CS 417/545 Project 1: Distributed Training

Due: 10/09/2026, 11:59 PM (100 points)

Please read this document before you start.

You will write a program that trains a small model the same way real systems train large ones. You split the work across a group of processes, sum the results, and repeat. You will do this three times, each version harder than the last.

There is no real machine learning here. The "model" is a slice of 16 floating-point numbers. The gradient is a library function we give you. The whole project is systems programming. What is hard is splitting the work correctly, moving data across a boundary, and keeping the result the same when a worker crashes.

Points are 20 (Part 1) + 15 (Part 2) + 65 (Part 3) = 100. CS 545 (graduate students) also do Section 8 (required for grads, worth an additional 15 points). Part 1 is one afternoon if you know Go. Part 2 is a few evenings and teaches you the goroutines and channels you need for Part 3. Part 3 is the main project.

Start early. See Section 10 for hints and Section 11 for the plagiarism and AI-use policy.

If you have not taken a machine learning class, read `p1-background.pdf` first. It explains the model, the dataset, the loss, and the gradient in plain language. It takes about 15 minutes. Everything past that document assumes you know these terms.

Code Structure

You get four files:

- `sealed.go` -- the model (`Init`, `Loss`, `Grad`), the data source (`NewBatcher`), and the RPC wire types. Do not modify this file. The grading server checks that your `sealed.go` matches ours before it runs anything.
- `p1.go` -- the file you edit. It has eight function signatures with `panic("not implemented yet")` bodies, plus a `main()` at the bottom that lets you run each part by hand. Do not change the function signatures or the fields inside any `Config` struct. You can add anything else you need.
- `yours_test.go` -- an empty test file with one smoke test and two commented-out templates. Write your own tests here as you go. Nothing here is graded.
- `go.mod` -- the module file. Standard library only. No third-party packages.

Section 4 lists what the tests check. Read it before you start on each part.

The first thing to do after you get the files:

```
go build .
go test -run TestExample .
```

If both work, your Go setup is fine and you can start Part 1.

Part 1. Sequential Trainer (20 points)

Part 1 is a Go warm-up. Read the four sealed functions in `sealed.go` first (`Init`, `Grad`, `Loss`, `NewBatcher`). Then implement:

```
func RunSeq(cfg SeqConfig) []float64
```

The loop is short. In pseudocode:

```
w := Init(cfg.Seed)
b := NewBatcher(cfg.Seed, cfg.BatchSize)
for step 0 .. Steps-1:
    batch := b.Next(step)
    g     := Grad(w, batch)                      // sum over the batch
    for i in 0..len(w)-1:
        w[i] -= cfg.LR * g[i] / cfg.BatchSize     // apply the update
    if cfg.OnStep != nil:
        cfg.OnStep(step, copyOf(w))
```

Return the final `w`.

Two things to watch for. First, `Grad` returns the sum of per-example gradients, not the average. You divide by `cfg.BatchSize` yourself in the update. Second, the slice you pass to `OnStep` must be a fresh copy. If you pass the trainer's own slice and then modify it later, the caller sees your changes. Make a copy.

Part 1 is where you learn enough Go to survive Part 2. If Part 1 is wrong, everything else is wrong.

Part 2. Parallel Local Trainer (15 points)

Now you run `NRanks` goroutines that work together on each step. Each goroutine (a "rank") has its own copy of the weights and computes the gradient on its slice of the batch. Every step ends with all ranks holding the same total gradient and applying the same update, so their copies of `w` stay the same. This is what real data-parallel training does, without the network.

Implement:

```
func RunLocal(cfg LocalConfig) []float64
```

`NRanks` must divide `BatchSize`. Rank `r` gets a contiguous slice of `BatchSize / NRanks` examples. Each rank sends its partial gradient to rank 0 through a channel. Rank 0 adds the partial gradients **in rank order** (rank 0 first, then rank 1, then rank 2, ...) and sends the total back on `NRanks` channels. Each rank applies the update. Return rank 0's final weights.

Why "in rank order" and not "in the order they arrive"? Because adding floating-point numbers in a different order can give a different result in the last bit. `(a+b)+c` and `a+(b+c)` are not always the same. If two runs of your trainer add in different orders, their final weights differ. Fixing the order fixes the problem. Real libraries like NCCL and MPI's `ALLREDUCE` do the same thing for the same reason.

Three rules the race detector will check:

- No shared memory between ranks. Talk through channels.
- No single mutex around the whole step.
- No rank reads another rank's weights.

The grading command uses `-race`, so any of these will show up when you run it too. Do not use `time.Sleep` for synchronization. It will bite you.

Part 3. Coordinator and Workers over RPC (65 points)

Part 3 splits Part 2 across processes. The coordinator holds the weights and sends a `Worker.Grad` RPC per slice to a pool of worker processes. Workers keep no state between calls. Everything a `Grad` call

needs is in its arguments. The reply depends only on those arguments. That is what makes the failure handling possible: any call can be sent again to any worker without changing the answer.

The RPC types are in `sealed.go`:

```
type GradArgs   struct { Step, SliceIndex int; W []float64; Batch []Example }
type GradReply  struct { Grad []float64 }
type PingArgs   struct{}
type PingReply  struct { OK bool }
```

The coordinator calls `"Worker.Grad"`. Register your service under that name.

3.1 Worker (10 points)

```
type Worker struct{}
func (Worker) Grad(args GradArgs, reply *GradReply) error
func (Worker) Ping(_ PingArgs, reply *PingReply) error
func RunWorker(addr string) (actualAddr string, stop func(), err error)
```

`Grad` checks that `len(args.W) == D` and `len(args.Batch) > 0`, and returns an error if not. On success it sets `reply.Grad = Grad(args.W, args.Batch)`. `Ping` sets `reply.OK = true`.

`RunWorker` opens a TCP listener on `addr`. Pass `"localhost:0"` to let the OS pick a free port. Register the `Worker` service on a fresh `rpc.NewServer()`. Start a goroutine that accepts connections in a loop. Return the actual listening address and a `stop()` function that closes the listener.

`stop()` must be safe to call more than once. If it is called twice and the second call panics, you have a bug. `sync.Once` is the usual way to fix this.

3.2 Coordinator (25 points)

```
func RunCoordinator(cfg DistConfig) []float64
```

Each step does this:

1. Draw the batch with `NewBatcher`, then split it into `cfg.NSlices` equal slices. `cfg.NSlices` must divide `cfg.BatchSize`. 2. Send a `Worker.Grad(w, slice_i)` RPC per slice, in parallel, across the live workers in your pool. 3. When every slice has a returned gradient, add them **in slice order** into `full`. Apply `w[i] -= cfg.LR * full[i] / cfg.BatchSize`. 4. If `cfg.OnStep != nil`, call it with a copy of `w`.

Adding in slice order is the same rule as Part 2, for the same reason. Do not add in the order the workers reply.

3.3 Failure Handling (30 points)

The grader will kill workers, slow them down, and refuse connections. Your coordinator must still finish. Rules:

- **Timeout.** If a slice's in-flight attempt has been running longer than `cfg.Timeout`, send a duplicate attempt to a different live worker. Do not cancel the first one; let it keep running.
- **First reply wins.** When any attempt returns a gradient, take that one and throw away the others for this slice.
- **Hard failure.** If an attempt returns an RPC error, drop that worker from the pool for good. There is no way to add it back.

- **All workers busy.** If every worker is running something else, wait until one returns. Do not spin. Do not panic.

Sending a duplicate attempt is safe because `Grad(w, slice)` is a pure function of its arguments.

Do not use `time.Sleep` to poll a timer. Use `time.After` or `time.NewTimer` (which you can reset).

Running workers on ilab. For Part 3 you need at least two machines so the coordinator and its workers are actually different processes on the network. See `iLab.pdf` for how to log in to ilab nodes, compile there, and pass one node's address to another. For this project it is enough to run all your workers on one extra ilab node while the coordinator runs on your primary one; you do not need a large ilab deployment.

4. What We Grade

Your code must handle the following cases.

Part 1.

- Two runs of `RunSeq` with the same `SeqConfig` return the same weights, bit for bit.
- With a few hundred steps, the loss on a held-out batch drops by at least 50x from the loss at `Init(Seed)`.
- Your final weights match a reference vector, bit for bit.
- If `cfg.OnStep` is set, you call it after every step with a fresh copy of the weights. Changing that copy inside the callback must not affect the trainer.

Part 2.

- `RunLocal` reduces loss for `NRanks` in `{1, 2, 4, 8}`.
- Two runs of `RunLocal` with the same `LocalConfig` return the same weights, bit for bit, at every value of `NRanks` we test.
- Your final weights at `NRanks = 4` match a reference vector, bit for bit.
- `go test -race` finds no data races.

Part 3.

- Healthy case: all workers alive. `RunCoordinator` returns weights whose loss dropped by at least 20x. Two runs with the same `DistConfig` return the same weights.
- Crashes: some workers close their listener partway through. Your final weights match the healthy run, bit for bit.
- Stragglers: some workers sleep a few hundred milliseconds before replying. Your final weights match the healthy run, bit for bit.

We do not tell you the exact `BatchSize`, `NSlices`, or `Timeout` values. Assume the ranges implied above (batch a small multiple of 8, slices a divisor of the batch, timeout in the low hundreds of milliseconds).

5. Running the Trainer by Hand

`main()` in `p1.go` uses `-mode`. Once each part compiles you can run:

```
go run . -mode seq
go run . -mode local -n 4
```

```
go run . -mode worker -addr localhost:9001 &
go run . -mode worker -addr localhost:9002 &
go run . -mode coord  -workers localhost:9001,localhost:9002
```

`seq` and `coord` print the loss every 25 steps and the final loss at the end. This is the fastest way to check that your code is doing something, before you write tests.

6. Testing Your Own Work

Write your own tests in `yours_test.go` as you build each part. A "loss goes down" check and a "two runs agree" check catch most Part 1 and Part 2 bugs. For Part 3, start workers with `RunWorker`, call `RunCoordinator`, kill a worker at step 5 using its `stop()` function, and check that the run finishes with sensible weights.

Run everything with the race detector:

```
go test -race .
```

Any race is a real bug. Do not ignore it.

If a test sometimes passes and sometimes fails, you have a race, a deadlock that shows up in some orderings, or a `time.Sleep` doing work that a channel should do.

7. Grading Policy

Points:

- **CS 417 (undergraduates):** 20 (Part 1) + 15 (Part 2) + 65 (Part 3) = 100.
- **CS 545 (graduate students):** the same 100 above, plus 15 more for Section 8 (required for grads). Your grade is scaled back to 100.

Part 3 breaks down as: 10 (Section 3.1, Worker) + 25 (Section 3.2, Coordinator) + 30 (Section 3.3, Failure Handling) = 65.

Partial credit is available. If your code compiles, shows serious effort, and works for most of the cases in Section 4, you will earn partial credit even if some tests fail. A single failed run of a graded test costs half of that test's points. The daily grading email is the final word.

8. Additional Work (Required for CS 545)

This section is required for CS 545 (graduate students) only. CS 417 (undergraduates) may implement any part of it for interest but it is not graded for undergraduates.

You will implement two more all-reduce variants and submit a short measurements report.

Ring all-reduce (5 points). Implement `RunRing(cfg LocalConfig) []float64` in `p1.go`. A "ring" is a topology, not a rank. Arrange the `NRanks` ranks in a logical ring. Each rank has a channel to its right neighbor and a channel from its left. In `NRanks - 1` accumulation hops, each rank adds its own gradient to the running sum from the left and sends it right. When the sum has gone around, every rank has the full total. There is no central rank, so no rank is a bottleneck.

Bucketed ring all-reduce (10 points). Implement `RunRingBucketed(cfg LocalConfig) []float64` in `p1.go`. Split the gradient into `NRanks` equal segments and pipeline them around the ring. While segment 0 is on its second hop, segment 1 is on its first, and so on. When every segment has gone around, every rank has the full sum, and the bandwidth per hop is `1/NRanks` of the plain ring. This is what real production libraries use.

Your `RunRing` and `RunRingBucketed` must return the same weights as `RunLocal` for the same `LocalConfig`, bit for bit. Same rule as before: reduction order.

Measurements. Put your measurements in `Report.md` (see Section 9). Time each version (`RunLocal`, `RunRing`, `RunRingBucketed`) over the same number of steps at `NRanks` of 2, 4, and 8. List the numbers. Explain in one paragraph why the bucketed version wins as the model gets larger.

9. Submission

Zip the whole `p1/` directory (skip any compiled binaries and any `.git/`) and upload to Canvas.

Include a `Report.md` in `p1/` with everything you want to tell us:

- Your name and NetID. If you worked with a partner, list them too.
- (CS 545 only) Your measurements from Section 8 (`RunLocal` vs `RunRing` vs `RunRingBucketed` at `NRanks` of 2, 4, 8) and the paragraph explaining why bucketed wins.
- Any AI tools you used and how; see Section 12.
- Anything else you want us to know before we grade.

`Report.md` is required for CS 545 (because of the measurements). For CS 417 it is required only if you have a partner or an AI-use disclosure. It is short: half a page is plenty.

Do not submit files from outside `p1/`.

10. Suggested Steps

1. Read `project1-desc.pdf` (this document) from top to bottom. If you have not taken a machine learning class, read `p1-background.pdf` next. 2. Skim `sealed.go` and `yours_test.go` so you know what you have before you write anything. 3. Run `go build .` on the skeleton. If it fails, fix your Go setup before you write any code. 4. Implement `RunSeq` (Part 1). Add a smoke test in `yours_test.go` that runs it and checks the returned length. 5. Add a second test that runs `RunSeq` twice with the same config and checks the two results match, bit for bit. When both tests pass, Part 1 is done. 6. Implement `RunLocal` (Part 2) at `NRanks = 1` first. That is just `RunSeq` wrapped in one extra layer of goroutine. Test it. 7. Generalize `RunLocal` to `NRanks = 2, 4, 8`. Draw the channel graph on paper for `NRanks = 3` before you code it. Watch for deadlocks and bit-determinism. Run with `go test -race .`. 8. Implement `Worker.Grad`, `Worker.Ping`, and `RunWorker` (Part 3.1). Confirm you can call the worker from a small standalone RPC client. 9. Implement `RunCoordinator` (Part 3.2) for the healthy case only, with all workers alive and no timeouts. Test that it reduces loss. 10. Add timeouts and duplicate attempts (Part 3.3). Run `go test -race .` several times in a row. A test that fails one time in ten is a race you have not found yet. Then submit.

CS 545 students: after step 10, add `RunRing`, `RunRingBucketed`, and put the measurements in `Report.md` (see Section 8 and Section 9).

11. Tips and Resources

Read failing tests, not the diff. The test name usually tells you which rule you broke. The diff of "got" and "want" usually does not.

Print `(step, rank, w[:3])` at the top of every step while you debug Part 2 or 3. Most bugs at this scale come from one rank being one step ahead of the others, and you cannot see that from the final weights.

Save a copy of your working directory after each part passes. If you break Part 3 with a bad edit, you want to go back without redoing Parts 1 and 2. `git commit` is fine. `cp -r p1 p1.bak` also works.

Go documentation you will hit often:

- `net/rpc`: <https://pkg.go.dev/net/rpc>
- `sync` (WaitGroup, Mutex, Once): <https://pkg.go.dev/sync>
- Race detector: <https://go.dev/doc/articles/race_detector>

The median time on a similar project last year was about nine hours. The tail was three to four times that. Plan for the tail.

12. Plagiarism and AI Use

This is individual work. You may talk with others about ideas. You must write your own code. Do not share your code, during or after the course.

AI tools (Copilot, Claude, ChatGPT, or similar) are allowed only as a reference when you get stuck on Go syntax or an error message you cannot parse. You cannot paste AI-generated code into your submission.

If you use AI tools, write a short note in `Report.md`:

1. Which tools you used. 2. Which parts of the project you used them on. 3. The prompts you used, word for word.

Honest disclosures earn a small participation bonus. Not disclosing is an academic integrity violation.